# 02-614 Week 8
String Algorithms

Aidan Jan

October 22, 2026

## Burrows-Wheeler Transform

Consider the string $S = banana\$$. We define burrows-wheeler transform (BWT) by first writing down every cyclic shifts of the string.

```
banana$
anana$b
nana$ba
ana$ban
na$bana
a$banan
$banana
```

Then, we arrange them in alphabetical order. Note that if arranged in alphabetic order, the $ comes first.

```
$banana
a$banan
ana$ban
anana$b
banana$
na$bana
nana$ba
```

The **last column** of the sorted list is the BWT. In this case, it will be $annb\$aa$.

- Note that even though *banana* does not have any repeating characters next to each other, there are repeating characters next to each other in the BWT.
- In fact, usually the BWT is more compressible than the original string.
  - This is because BWT can be thought of as a dictionary, where the last character in the string is the *value* and everything preceding it is the *key*.
  - This means that the order of values are sorted by the part that occurs directly after it (because cyclic shifts). If we have a substring that appears multiple times, it becomes highly compressible.

### Inverting BWT

Miraculously, BWT is invertible, meaning that given just the BWT of the string, we can reconstruct the original string. Consider *banana*$ again. We start with the last column.

```
------a
------n
------n
------b
```

```
------$
------a
------a
```

We know that the first column of the BWT matrix is all the characters in sorted order. So, we can just fill that in.

```
$-----a
a-----n
a-----n
a-----b
b-----$
n-----a
n-----a
```

Now, observe that if we imagine a cyclic shift, the first string tells us that $a$\$ is a substring. The following rows tell us that $na$ is a substring (that appears twice), $ba$ is a substring, etc. We now have all the substrings of length 2. Now, we repeat the same logical step as before; the first two columns of BWT must be the all the length 2 substrings in sorted order.

```
$b----a
a$----n
an----n
an----b
ba----$
na----a
na----a
```

We can repeat this process until we have the entire original string.

## Constructing BWT in Linear Time

Note that the first step of BWT is to sort all the cyclic shifts. However, we don't actually have to sort all the cyclic shifts.

Due to the nature of cyclic shifts always containing exactly one terminus character, sorting the shifts will yield the same ordering as sorted suffixes.

- We already have the suffixes in sorted order, in linear time. That's a suffix array!

Now, getting the BWT column (the last column) are just the cahracter preceeding each of the suffixes. We can find that in constant time if given the ordering.

## Inverting BWT in Linear Time

Earlier we discussed an algorithm to invert BWT, but it takes quadratic time. This is because the matrix of letters we fill out is the length of the string squared. Here, we will get the original string from the BWT in linear time.

Consider the string $S = dogwood$\$. Below is the BWT matrix for the string, already in sorted order.

```
$dogwood
d$dogwoo
dogwood$
gwood$do
od$dogwo
ogwood$d
ood$dogw
wood$dog
```

**LF Mapping**

**Theorem:** For any $c$ the $i$-th occurrance of $c$ in $F$ is the same as the $i$-th occurrence of $c$ in BWT($S$).

- $F$ refers to the first column of the matrix, and $L$ refers to the last column of the matrix. $L$ in this case is another name for BWT($S$).
- Proof: For any $c$, sorting in $L$ and $F$ use the same set of keys.

We can now add numbers to each row of the matrix:

```
1 $dogwood 1
1 d$dogwoo 1
2 dogwood$ 1
1 gwood$do 2
1 od$dogwo 3
2 ogwood$d 2
3 ood$dogw 1
1 wood$dog 1
```

The numbers on the left represent the number of times the first character has appeared as the first character. (i.e., in the second row, `d` is labeled as 1 because it is the first time `d` appears in the first column. In row three, it is labeled as 2 because it is the second time $d$ appears in the first column.) The numbers on the right are the same but instead for the last character of the string.

What the LF mapping theorem is saying is that the first occurrence of a given character in the $F$ column is the same character's first occurrence in the $L$ column.

- For example, the first $o$ in $F$ is the same character in the string as the first $o$ in $L$.

---

To invert faster, we first generate the first column using the BWT, along with their numberings.

Additionally, create an array the size of our alphabet, and let each array index correspond to the number of times a given character (in alphabetical order) appears.

- This takes $O(n)$ time to iterate through the BWT column and count the number of appearances of each character
- This array can then be used to generate the first column in linear time instead of sorting the last column.

```
1 $------d 1
1 d------o 1
2 o------$ 1
1 g------o 2
1 o------o 3
2 o------d 2
3 o------w 1
1 w------g 1
```

Recall that the first column is the characters of the last column in alphabetical order. Now, to invert, start with the first character in $L$. That must be the last character since it preceeds \$ in $F$.

```
------d$
```

Now, use the LF mapping property. The last character we added was the first `d`, so we go to its row in $F$. Using the predecessor property (same as the one for the naive inversion method), we know the first `o` precedes the first `d`. Therefore, we can fill it in:

```
-----od$
```

We can repeat this process, alternating the LF mapping property and predecessor property to derive the original string.

## BWT Search

Consider $S = abbbaba$, and $P = abb$. We want to determine whether $P$ exists in $S$, and how many times it occurs. (We can't find the locations)

```
row | L          R
 0  | 1 $------a 1
 1  | 1 a------b 1
 2  | 2 a------b 2
 3  | 3 a------$ 1
 4  | 1 b------a 2
 5  | 2 b------b 3
 6  | 3 b------b 4
 7  | 4 b------a 3
```

## Compression

Suppose we want to compress our string. Let $S = abbaaaa$. To demonstrate the compression method, we first imagine a compressor and decompressor, $C$ and $D$. The two arrays are first initialized to empty arrays.

| Character | $\Sigma_C$ | Packet | $\Sigma_D$ |
|---|---|---|---|
| (Init) | [ ] | | [ ] |

The goal of the compressor is to send the string to the decompressor, one character at a time. First, we consider the first $a$. $C$ it in its array, sends over the character to $D$, along with an index which represents the character's position in $\Sigma_C$.

| Character | $\Sigma_C$ | Packet | $\Sigma_D$ |
|---|---|---|---|
| (Init) | [ ] | | [ ] |
| $a$ | $[a]$ | (a, 0) | $[a]$ |

Now, we consider the second character in our string, $b$. Because it does not exist yet in $\Sigma_C$, we append it in the beginning and send the character and index to $D$.

| Character | $\Sigma_C$ | Packet | $\Sigma_D$ |
|---|---|---|---|
| (Init) | [ ] | | [ ] |
| $a$ | $[a]$ | (a, 0) | $[a]$ |
| $b$ | $[b, a]$ | (b, 0) | $[b, a]$ |

No space for compression yet, but here, things get interesting. The next character is the second occurrence of $b$. Because it is already present in the array, we don't have to send the character again.

| Character | $\Sigma_C$ | Packet | $\Sigma_D$ |
|---|---|---|---|
| (Init) | [ ] | | [ ] |
| $a$ | $[a]$ | (a, 0) | $[a]$ |
| $b$ | $[b, a]$ | (b, 0) | $[b, a]$ |
| $b$ | $[b, a]$ | 0 | $[b, a]$ |

Now, we have the second $a$. In this case, it is the first index of the array. However, whenever we get a character that is not already at the front, we move it to the front in the array. Therefore, sending $a$ again would place it at the front of the array.

| Character | $\Sigma_C$ | Packet | $\Sigma_D$ |
|---|---|---|---|
| (Init) | [ ] | | [ ] |
| $a$ | $[a]$ | (a, 0) | $[a]$ |
| $b$ | $[b, a]$ | (b, 0) | $[b, a]$ |
| $b$ | $[b, a]$ | 0 | $[b, a]$ |
| $a$ | $[a, b]$ | 1 | $[a, b]$ |

For the rest of the $a$, each row we only send a zero since it is the first character of the list already.

- The idea with the swapping characters is that (recently) common characters will be at the beginning of the array, and uncommon characters will slowly be pushed into the back.
- Smaller numbers take less bits to send than big numbers.

#### Prefix Encoding

Suppose we want to encode character $i$ with $0000001\ldots$, where we have $\log(i)$ number of 0's at the beginning. [FILL]

#### Run-Length Encoding

Suppose we need to send a series of one million zeros. $00000\ldots$. Instead of sending one million zeros, we just send $(1000000, 0)$.

## Rank / Select

Earlier, we introduced the concept of rank implicitly.

$$\text{Rank}_1(i) = \text{Number of 1's before position } i$$